

Module 3: SQL Query Results

Robby Ketchell · EN.605.256 · generated 2026-10-02 from the live PostgreSQL **applicants** table using query_data.py (psycopg).

Formatting: counts are whole numbers, percentages have two decimal places and a % sign, averages have two decimal places. Averages include only applicants who report that metric. All text matching is case-insensitive.

Question 1

How many entries in the database are from applicants who applied for Fall 2026?

Result

Fall 2026 applicant count: 33,207

SQL

```
SELECT COUNT(*) AS "Fall 2026 applicant count"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026';
```

Explanation

Counts every row whose term field is 'Fall 2026'. TRIM and LOWER make the comparison robust to stray whitespace and capitalization differences.

Question 2

Among entries that provide a nationality classification, what percentage are international students?

Result

Percent international: 47.89%

SQL

```
SELECT ROUND(
    100.0 * COUNT(*) FILTER (WHERE LOWER(TRIM(us_or_international)) = 'international')
    / NULLIF(COUNT(*), 0),
    2) AS "Percent international"
FROM applicants
WHERE us_or_international IS NOT NULL
    AND TRIM(us_or_international) <> '';
```

Explanation

The WHERE clause restricts the denominator to rows with a usable nationality value (NULL and blank strings are excluded). The FILTER clause counts only the rows labelled 'International' for the numerator, so 'American' and 'Other' are in the denominator but not the numerator. NULLIF avoids division by zero.

Question 3

What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

Result

Average GPA: 3.75

Average GRE Quantitative: 165.74

Average GRE Verbal: 160.37

Average GRE Analytical Writing: 4.33

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS "Average GPA",  
       ROUND(AVG(gre)::numeric, 2) AS "Average GRE Quantitative",  
       ROUND(AVG(gre_v)::numeric, 2) AS "Average GRE Verbal",  
       ROUND(AVG(gre_aw)::numeric, 2) AS "Average GRE Analytical Writing"  
FROM applicants;
```

Explanation

AVG() ignores NULLs, so each column's average is computed only over the rows that supply that particular metric. An applicant with a GPA but no GRE still contributes to the GPA average, which is exactly what the assignment asks for.

Question 4

What is the average GPA of American applicants who applied for Fall 2026?

Result

Average GPA (American, Fall 2026): 3.79

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS "Average GPA (American, Fall 2026)"  
FROM applicants  
WHERE LOWER(TRIM(term)) = 'fall 2026'  
      AND LOWER(TRIM(us_or_international)) = 'american'  
      AND gpa IS NOT NULL;
```

Explanation

Filters to Fall 2026 rows classified as American that also report a GPA, then averages that GPA. The explicit gpa IS NOT NULL mirrors the requirement even though AVG() would skip NULLs anyway.

Question 5

What percentage of Fall 2025 entries are acceptances?

Result

Fall 2025 acceptance percentage: 40.74%

SQL

```
SELECT ROUND(
    100.0 * COUNT(*) FILTER (WHERE status ILIKE 'accept%')
    / NULLIF(COUNT(*), 0),
    2) AS "Fall 2025 acceptance percentage"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2025';
```

Explanation

All Fall 2025 rows form the denominator; the FILTER clause counts the subset whose cleaned status starts with 'Accept' (the cleaned status values are Accepted / Rejected / Wait listed / Interview) for the numerator.

Question 6

What is the average GPA of accepted applicants who applied for Fall 2026?

Result

Average GPA (accepted, Fall 2026): 3.76

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS "Average GPA (accepted, Fall 2026)"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND status ILIKE 'accept%'
    AND gpa IS NOT NULL;
```

Explanation

Same shape as Question 4, but the nationality filter is replaced by an acceptance filter on the status column.

Question 7

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

Result

JHU Computer Science master's entries: 18

SQL

```
SELECT COUNT(*) AS "JHU Computer Science master's entries"
FROM applicants
WHERE (program ILIKE '%johns hopkins%' OR program ~* '\mjhu\M')
      AND program ILIKE '%computer science%'
      AND (degree ILIKE '%master%' OR degree ~* '^m\.?s(c|\.)?$');
```

Explanation

Uses the original program and degree fields. The program text holds both the department and the university, so ILIKE matches 'Johns Hopkins' anywhere in it, and the \m...\M regex matches 'JHU' as a whole word. 'Computer Science' is matched as a substring, and the degree must be a master's variant (Masters, MS, M.S., MSc).

Question 8

How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown, MIT, Stanford, or Carnegie Mellon (original fields)?

Result

Original-field count: 30

SQL

```
SELECT COUNT(*) AS "Original-field count"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
      AND status ILIKE 'accept%'
      AND degree ~* 'ph\.?s?d'
      AND program ILIKE '%computer science%'
      AND (
        program ILIKE '%georgetown%'
        OR program ILIKE '%massachusetts institute of technology%'
        OR program ~* '\mmit\M'
        OR program ILIKE '%stanford%'
        OR program ILIKE '%carnegie mellon%'
        OR program ~* '\mcmu\M'
      );
```

Explanation

All five restrictions are ANDed together: the term, an accepted status, a PhD degree (PhD / Ph.D.), 'Computer Science' somewhere in the original program text, and the original program text naming one of the four universities (full names plus the MIT / CMU abbreviations as whole words).

Question 9

Repeat Question 8 using llm_generated_program and llm_generated_university for the program and university, and report both counts and their difference.

Result

Original-field count: 30

LLM-field count: 30

Difference: +0

SQL

```
WITH base AS (
  SELECT program, llm_generated_program, llm_generated_university
  FROM applicants
  WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND status ILIKE 'accept%'
    AND degree ~* 'ph\.\?\s?d'
),
original AS (
  SELECT COUNT(*) AS n FROM base
  WHERE program ILIKE '%computer science%'
    AND (
      program ILIKE '%georgetown%'
      OR program ILIKE '%massachusetts institute of technology%'
      OR program ~* '\mit\M'
      OR program ILIKE '%stanford%'
      OR program ILIKE '%carnegie mellon%'
      OR program ~* '\mcmu\M'
    )
),
llm AS (
  SELECT COUNT(*) AS n FROM base
  WHERE llm_generated_program ILIKE '%computer science%'
    AND LOWER(TRIM(llm_generated_university)) IN (
      'georgetown university',
      'massachusetts institute of technology',
      'stanford university',
      'carnegie mellon university'
    )
)
SELECT original.n          AS "Original-field count",
       llm.n              AS "LLM-field count",
       llm.n - original.n AS "Difference"
FROM original, llm;
```

Explanation

The CTE 'base' applies the filters that stay on the original fields (term, status, degree). 'original' re-applies the Question 8 program/university matching, while 'llm' matches the LLM-standardized program with ILIKE and the LLM-standardized university with exact (case-insensitive) names. The final SELECT reports both counts side by side and their signed difference.

Why the counts may or may not differ: the two queries only disagree where the raw program text and the LLM-standardized fields describe the same entry differently. The original-field match depends on applicants typing a recognisable university name ('MIT', 'Carnegie Mellon', 'Stanford University') and the words 'Computer Science' somewhere in the program string, so an entry written as 'CS, Stanford' or with a misspelling would be missed. The LLM field collapses spelling and abbreviation variants onto canonical names, which can add entries the raw match missed, but it can also drop entries when

the model mislabels the university or rewrites the program (for example turning 'EECS' into a name without the phrase 'Computer Science') or hallucinates a different school for an ambiguous string. In this dataset the entries for these four highly ranked programs are already written cleanly on Grad Cafe, so both approaches select the same rows and the difference is zero. With messier free text the LLM count would usually be the larger of the two.

Question 10

Original question 1: For Fall 2026, do American and international applicants report different acceptance rates?

Result

Nationality	Entries	Accepted	Acceptance percent
American	17,726	6,510	36.73%
International	14,791	5,066	34.25%

SQL

```
SELECT us_or_international AS "Nationality",
       COUNT(*) AS "Entries",
       COUNT(*) FILTER (WHERE status ILIKE 'accept%') AS "Accepted",
       ROUND(100.0 * COUNT(*) FILTER (WHERE status ILIKE 'accept%') / COUNT(*), 2) AS "Acceptance percent"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
      AND us_or_international IS NOT NULL
      AND TRIM(us_or_international) <> ''
GROUP BY us_or_international
ORDER BY "Entries" DESC;
```

Explanation

Groups the Fall 2026 rows that carry a nationality label and, for each group, counts the entries, counts the acceptances with a FILTER clause, and turns the ratio into a percentage. Comparing the rows shows whether the two populations report different acceptance rates on Grad Cafe.

Question 11

Original question 2: Which ten universities have the most Fall 2026 entries, and what are their acceptance percentages and average reported GPAs?

Result

University	Entries	Acceptance percent	Average GPA
Stanford University	770	18.44%	3.85
University of California, Berkeley	722	25.48%	3.83
Yale University	633	14.22%	3.84
University of Washington	623	25.84%	3.76
University of Wisconsin–Madison	604	20.03%	3.81
Princeton University	597	20.60%	3.86
University of Michigan, Ann Arbor	584	25.68%	3.82
University of Chicago	579	27.12%	3.82
Massachusetts Institute of Technology	574	18.64%	3.85
University of Toronto	534	39.70%	3.79

SQL

```
SELECT llm_generated_university AS "University",
       COUNT(*) AS "Entries",
       ROUND(100.0 * COUNT(*) FILTER (WHERE status ILIKE 'accept%') / COUNT(*), 2) AS "Acceptance percent",
       ROUND(AVG(gpa)::numeric, 2) AS "Average GPA"
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
      AND llm_generated_university IS NOT NULL
GROUP BY llm_generated_university
ORDER BY COUNT(*) DESC
LIMIT 10;
```

Explanation

Uses the LLM-standardized university name so that spelling variants collapse into one group. Groups the Fall 2026 rows by university, orders the groups by size and keeps the ten largest, reporting each group's entry count, acceptance percentage and the average GPA among entries that report one.